

Autonomous Vehicle Intersection Management System

A signal-free 4-way intersection, coordinated by a verified roadside controller — from UML models models to a proven UPPAAL design to running FreeRTOS code.

Tawhidur Rahman Tuhin

Moshiur Rahman Prince

Md Ratul Ahmed Alvi

The Problem

A 4-way intersection with no traffic lights — every vehicle is autonomous, and a roadside computer decides who crosses.

- Cars announce position and intent over V2V and V2I radio links.
- No two conflicting trajectories may ever be inside the box together.
- Requests are served first-in-first-served; no entry without permission.
- **A primary and a backup coordination server remove the single point of failure.**

THE THREE QUESTIONS

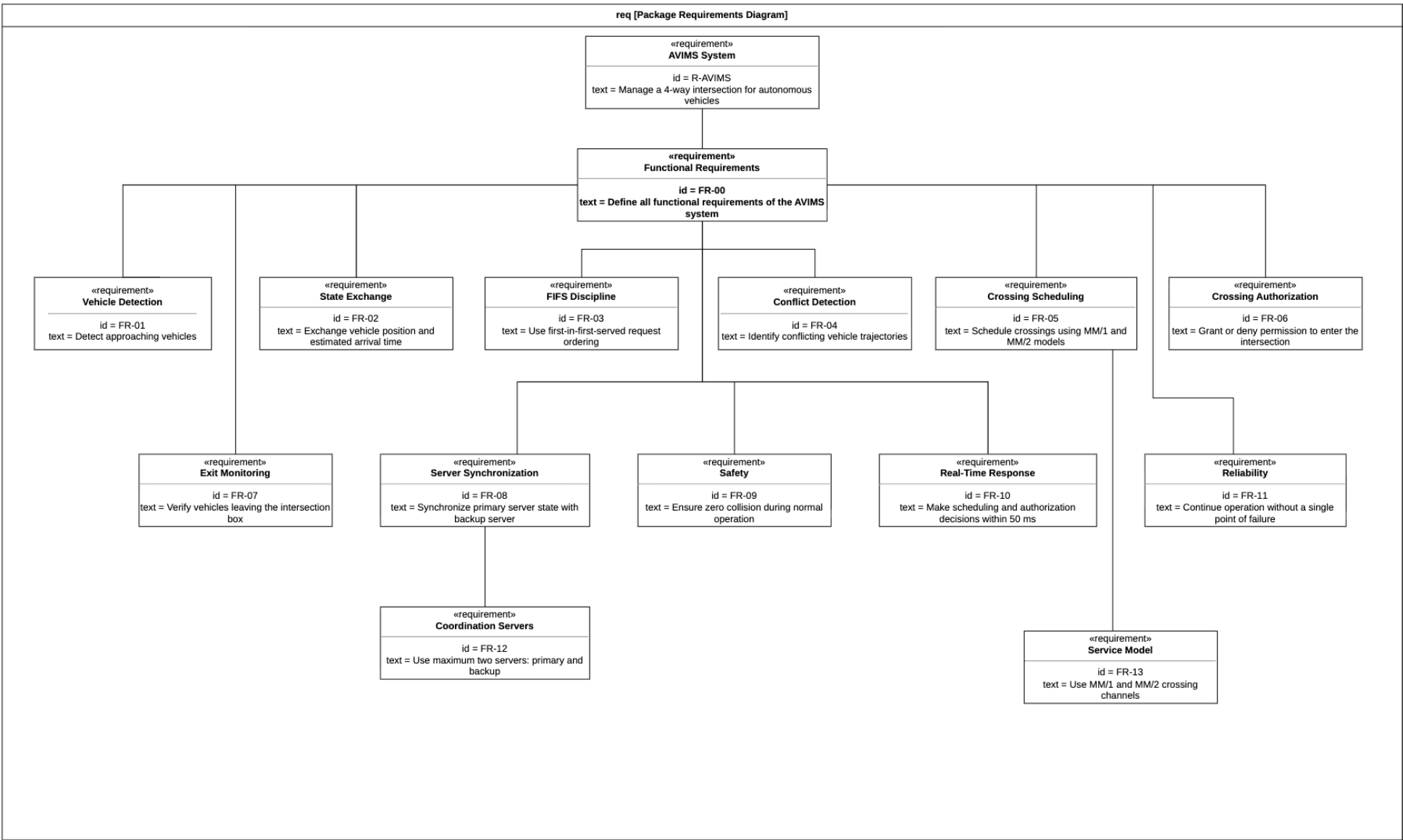
- 1 How is intersection traffic modeled with queuing theory?
 - 2 Which strategy best reduces congestion at the crossing?
 - 3 How is it simulated on freeRTOS, with part of it in hardware?
-

Requirements Diagram

The vehicle pipeline: detect, exchange state, queue FIFS, check conflicts, schedule, authorize, monitor exit.

The hard bounds: zero collisions, and every decision within 50 ms.

Fault tolerance: primary state synced to to a backup server — at most two, no single point of failure.

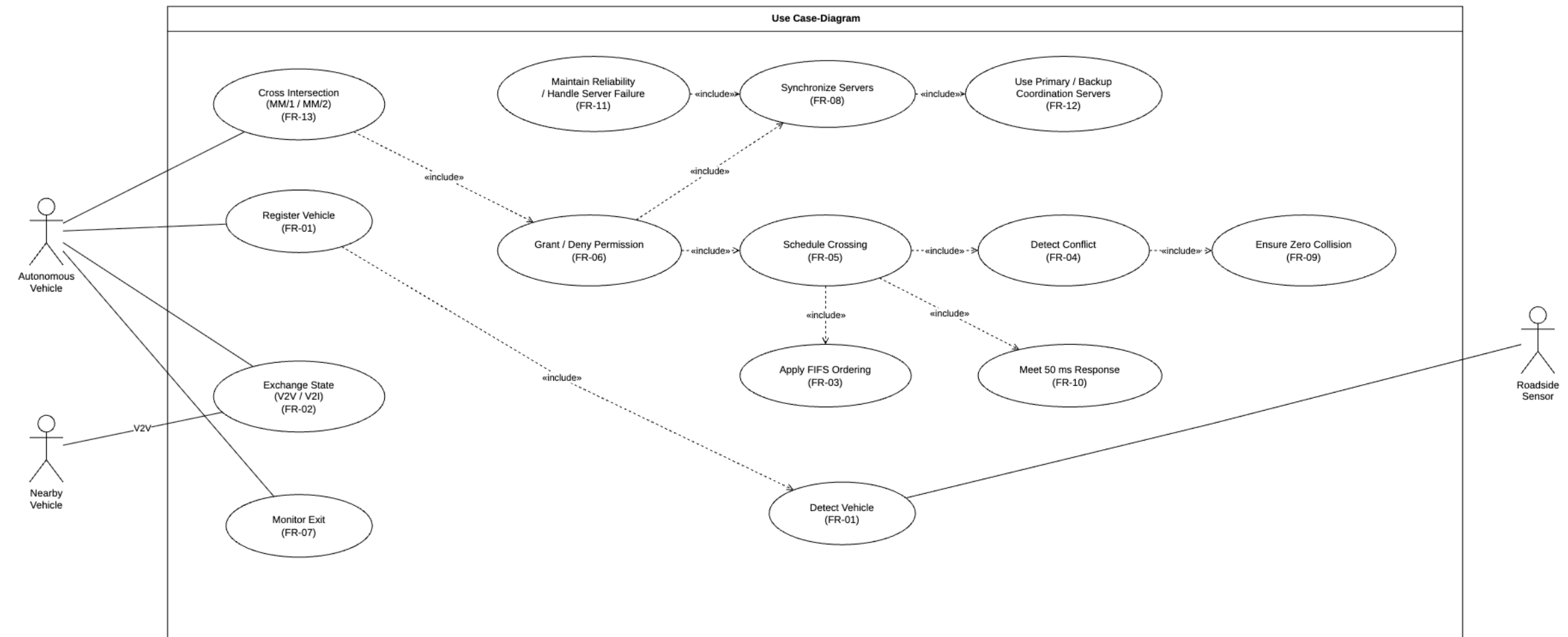


Use Case Diagram

Autonomous vehicle, nearby vehicle (V2V), roadside sensor — they use the system.

Primary and backup coordination servers are the system, not external users of it.

Cross → grant → schedule → detect conflict, under FIFS ordering and the 50 ms bound.

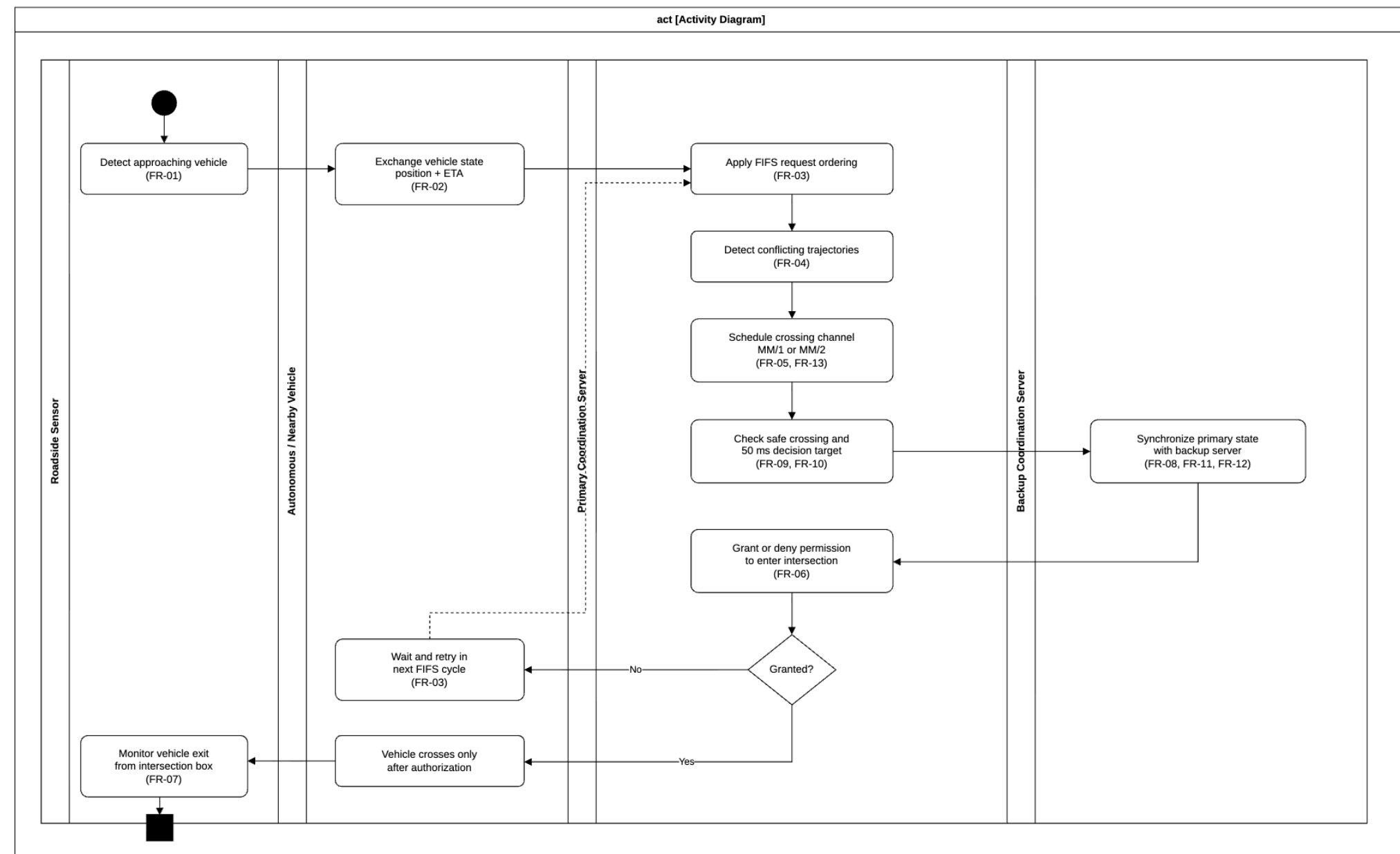


Activity Diagram

FIFS order → conflict check → M/M/1
or M/M/2 channel → 50 ms budget.

The backup server is updated before
any permission is issued.

A denied car waits and re-enters the
next FIFS cycle — nobody is dropped.

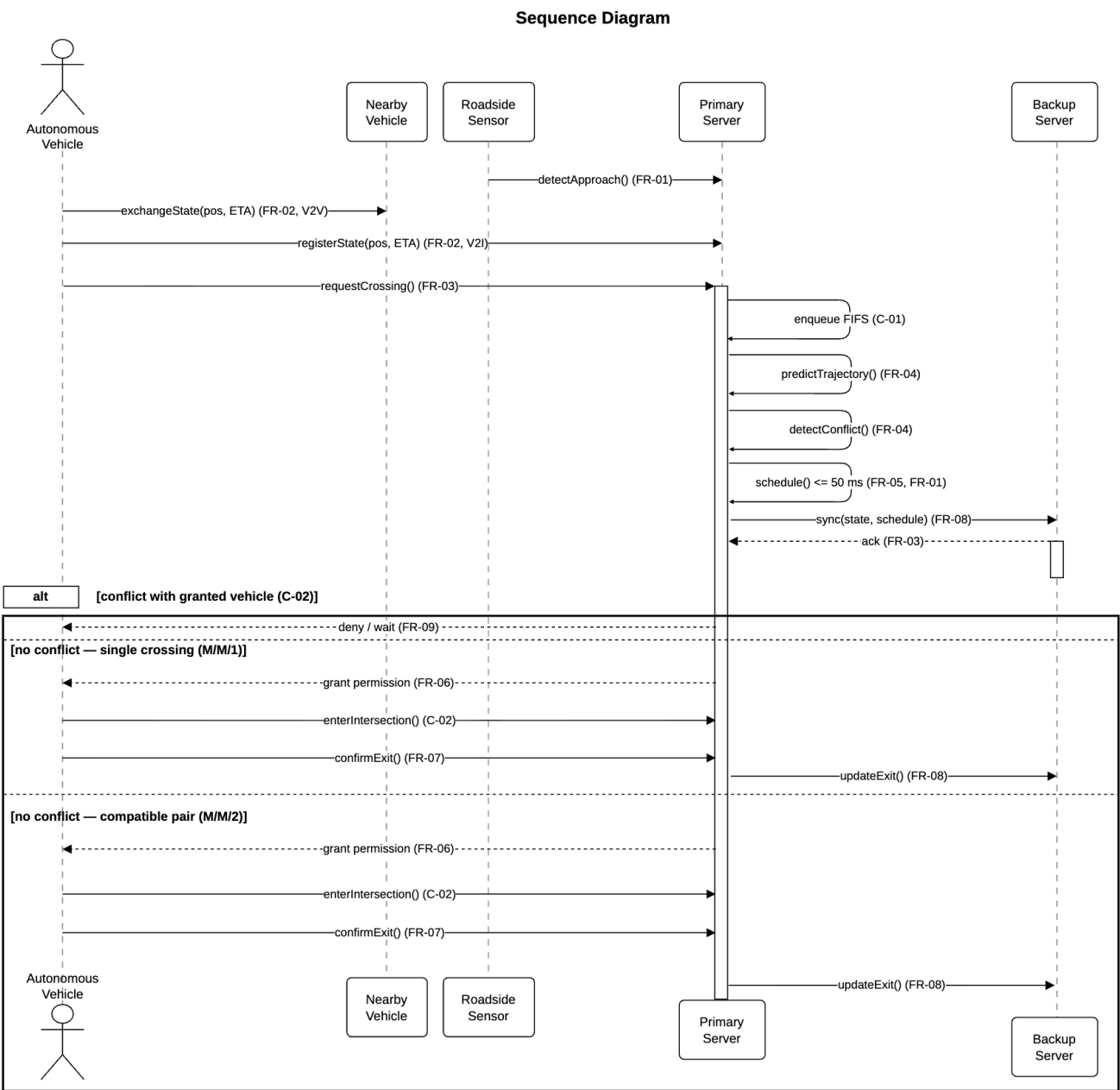


Sequence Diagram

enqueue FIFS → predictTrajectory →
detectConflict → schedule ≤ 50 ms.

State and schedule are pushed to the
backup, and acknowledged, before the
grant.

Conflict → deny · no conflict → M/M/1
crossing · compatible pair → M/M/2
together.



Model Structure

Vehicle

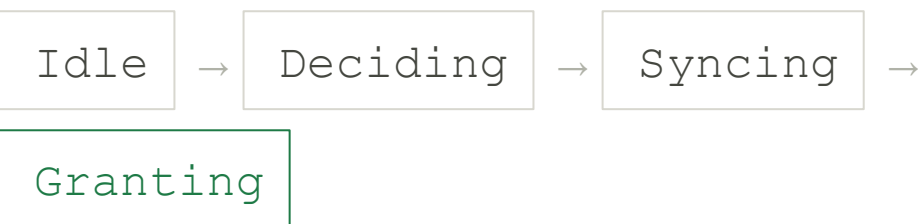
× 4



Clock travel: inside the box between 20 and 80 ms — enforced by invariant + guard.

PrimaryServer

× 1



Clock decide ≤ 50 ms — the FR-10 real-time time budget, checked on every cycle.

BackupServer

× 1



Receives sync!, answers ack! — the primary cannot grant until the handshake completes.

6 automata in parallel

channels: request / sync / ack / grant

const CHANNELS = 1 | 2 — the M/M/1 ↔ M/M/2 switch

M/M/1 vs M/M/2

M/M/1 · CHANNELS = 1

1

car crosses at a time strictly serialized

- Occupancy never exceeds 1, even for compatible pairs
- Queue grows under load
- property 7 proven

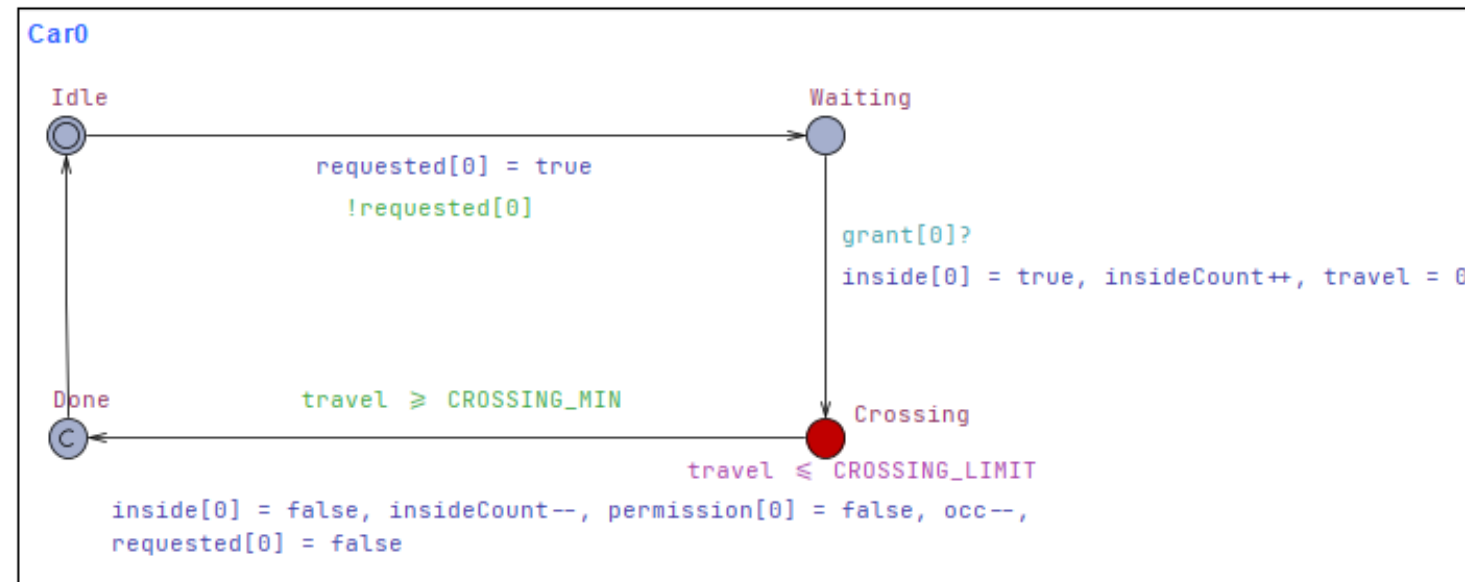
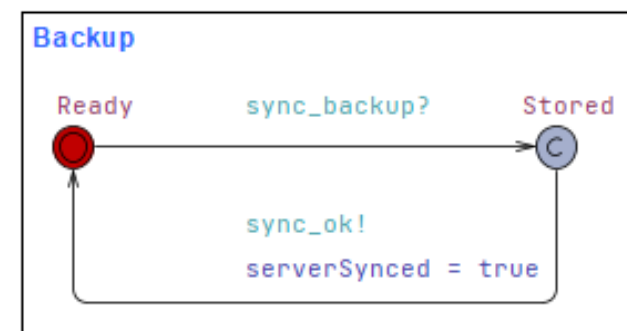
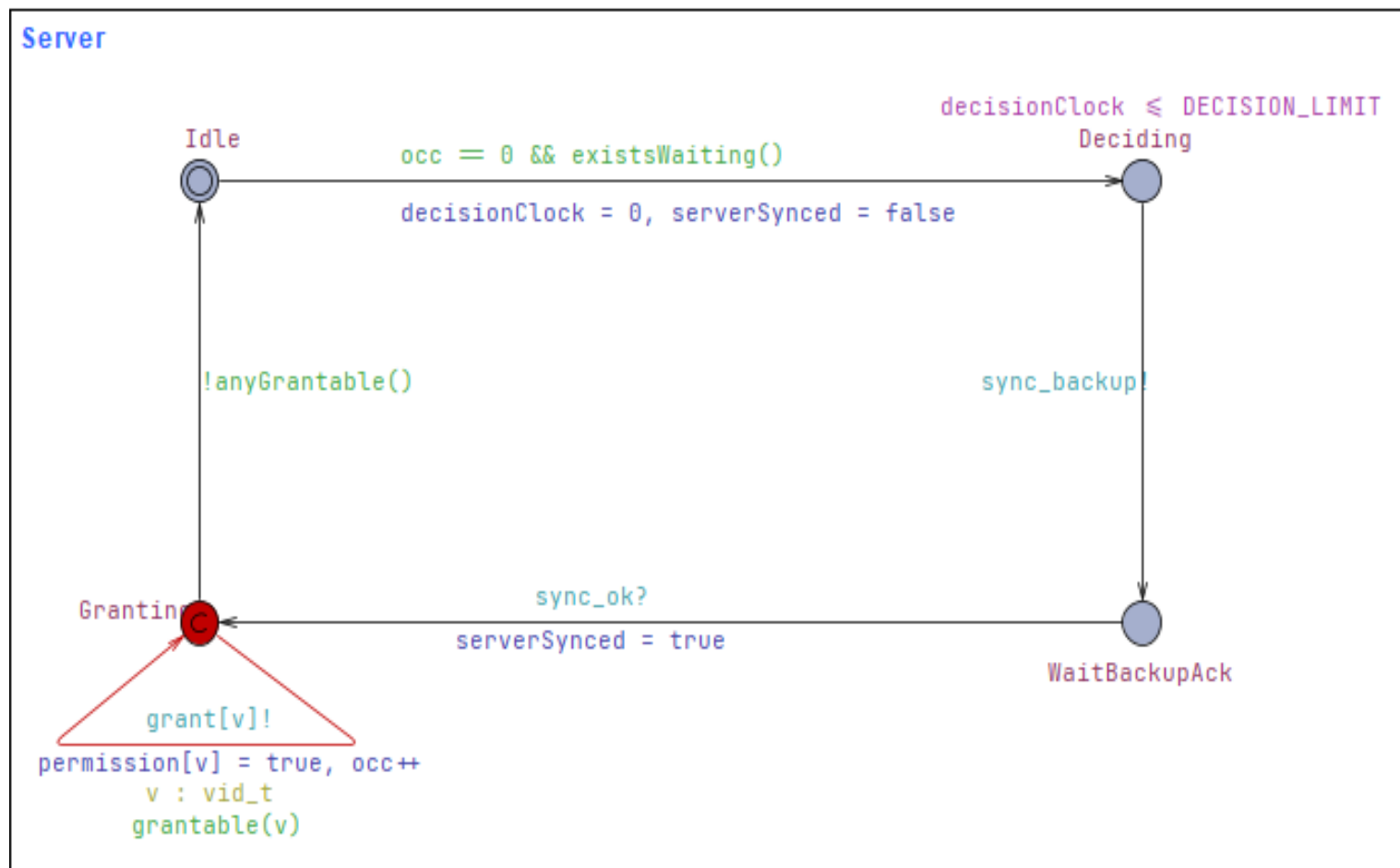
M/M/2 · CHANNELS = 2

2

cross together when the matrix says compatible

- Compatible pairs (e.g. NS + SS) admitted simultaneously
- Shorter queues, higher throughput - congestion reduced
- All 7 properties still satisfied - safety costs nothing

Model Structure



6 automata in
parallel

const CHANNELS = 1 | 2 – the M/M/1 ↔
M/M/2 switch

Verified Properties

Seven safety and liveness queries, model-checked exhaustively in UPPAAL.

1 Deadlock freedom

```
A[] not deadlock
```

2 Collision safety

```
A[] not (inside[i] && inside[j] && conflict)
```

3 No entry without a grant

```
A[] inside[i] imply permission[i]
```

4 50 ms decision bound

```
A[] Deciding imply clk <= 50ms
```

5 Sync before grant

```
A[] permission[i] imply serverSynced
```

6 Bounded occupancy

```
A[] occ <= CHANNELS
```

7 Simultaneous crossing

```
8 E<> inside[0] && inside[1]
```

Overview	
A[] not deadlock	
A[] forall (i : vid_t) forall (j : vid_t) (i ≠ j && inside[i] && inside[j]) imply !conflictMoves(move_of[i], move_of[j])	
A[] forall (i : vid_t) inside[i] imply permission[i]	
A[] (Server.Deciding imply decisionClock ≤ DECISION_LIMIT)	
A[] forall (i : vid_t) permission[i] imply serverSynced	
A[] (occ ≤ CHANNELS && insideCount ≤ CHANNELS)	
E<> (inside[0] && inside[1])	

UPPAAL verifier — every query returns satisfied (green check, right column).

Properties 1–5 hold for both M/M/1 and M/M/2; property 7 confirms the M/M/2 throughput path.

The Conflict Matrix

N

Conflict matrix

Hover a cell to see the two paths. ■ conflict ■ complete

	NR	NS	NL	ER	ES	EL	SR	SS	SL	WR	WS	WL
NR	–	1	1	0	1	0	0	0	1	0	0	0
NS	1	–	1	0	1	1	0	0	1	1	1	1
NL	1	1	–	0	1	1	1	1	0	0	1	1
ER	0	0	0	–	1	1	0	1	0	0	0	1
ES	1	1	1	1	–	1	0	1	1	0	0	1
EL	0	1	1	1	1	–	0	1	1	1	1	0
SR	0	0	1	0	0	0	–	1	1	0	1	0
SS	0	0	1	1	1	1	1	–	1	0	1	1
SL	1	1	0	0	1	1	1	1	–	0	1	1
WR	0	1	0	0	0	1	0	0	0	–	1	1
WS	0	1	1	0	0	1	1	1	1	1	–	1
WL	0	1	1	1	1	0	0	1	1	1	1	–

S

W

E

```
architecture simple of conflict_rom is
```

```
    type conflict_table_type is array (0 to 11, 0 to 11) of std_logic;
```

```
    -- columns: NR NS NL ER ES EL SR SS SL WR WS WL
```

```
    constant CONFLICT_TABLE : conflict_table_type := (
```

```
        ( '0', '1', '1', '0', '1', '0', '0', '0', '1', '0', '0', '0', '0' ), -- row 0 : NR
```

```
        ( '1', '0', '1', '0', '1', '1', '0', '0', '1', '1', '1', '1', '1' ), -- row 1 : NS
```

```
        ( '1', '1', '0', '0', '1', '1', '1', '1', '0', '0', '1', '1', '1' ), -- row 2 : NL
```

```
        ( '0', '0', '0', '0', '1', '1', '0', '1', '0', '0', '0', '0', '1' ), -- row 3 : ER
```

```
        ( '1', '1', '1', '1', '0', '1', '0', '1', '1', '1', '0', '0', '1' ), -- row 4 : ES
```

```
        ( '0', '1', '1', '1', '1', '0', '0', '1', '1', '1', '1', '1', '0' ), -- row 5 : EL
```

```
        ( '0', '0', '1', '0', '0', '0', '0', '1', '1', '1', '0', '1', '0' ), -- row 6 : SR
```

```
        ( '0', '0', '1', '1', '1', '1', '1', '0', '1', '0', '1', '1', '1' ), -- row 7 : SS
```

```
        ( '1', '1', '0', '0', '1', '1', '1', '1', '0', '0', '1', '1', '1' ), -- row 8 : SL
```

```
        ( '0', '1', '0', '0', '0', '1', '0', '0', '0', '0', '0', '1', '1' ), -- row 9 : WR
```

```
        ( '0', '1', '1', '0', '0', '1', '1', '1', '1', '1', '1', '0', '1' ), -- row 10 : WS
```

```
        ( '0', '1', '1', '1', '1', '0', '0', '1', '1', '1', '1', '1', '0' ) -- row 11 : WL
```

```
    );
```

FreeRTOS Task Architecture

Unit-tested standalone
randomized rounds, zero safety violations.
violations.

Core Components

xQueueCreate / Send / Receive –
task mailboxes

xTaskNotify – the “go” signal to
workers

xSemaphoreCreateMutex – clean event
output

vTaskDelay – the pausable crossing
timer

SchedulerTask	<code>prio 4</code>	sole owner of the core — decides within 50 ms, syncs backup, dispatches grants
----------------------	---------------------	--

BackupTask	<code>prio 3</code>	the second coordination server — stores the sync, returns the ack
-------------------	---------------------	---

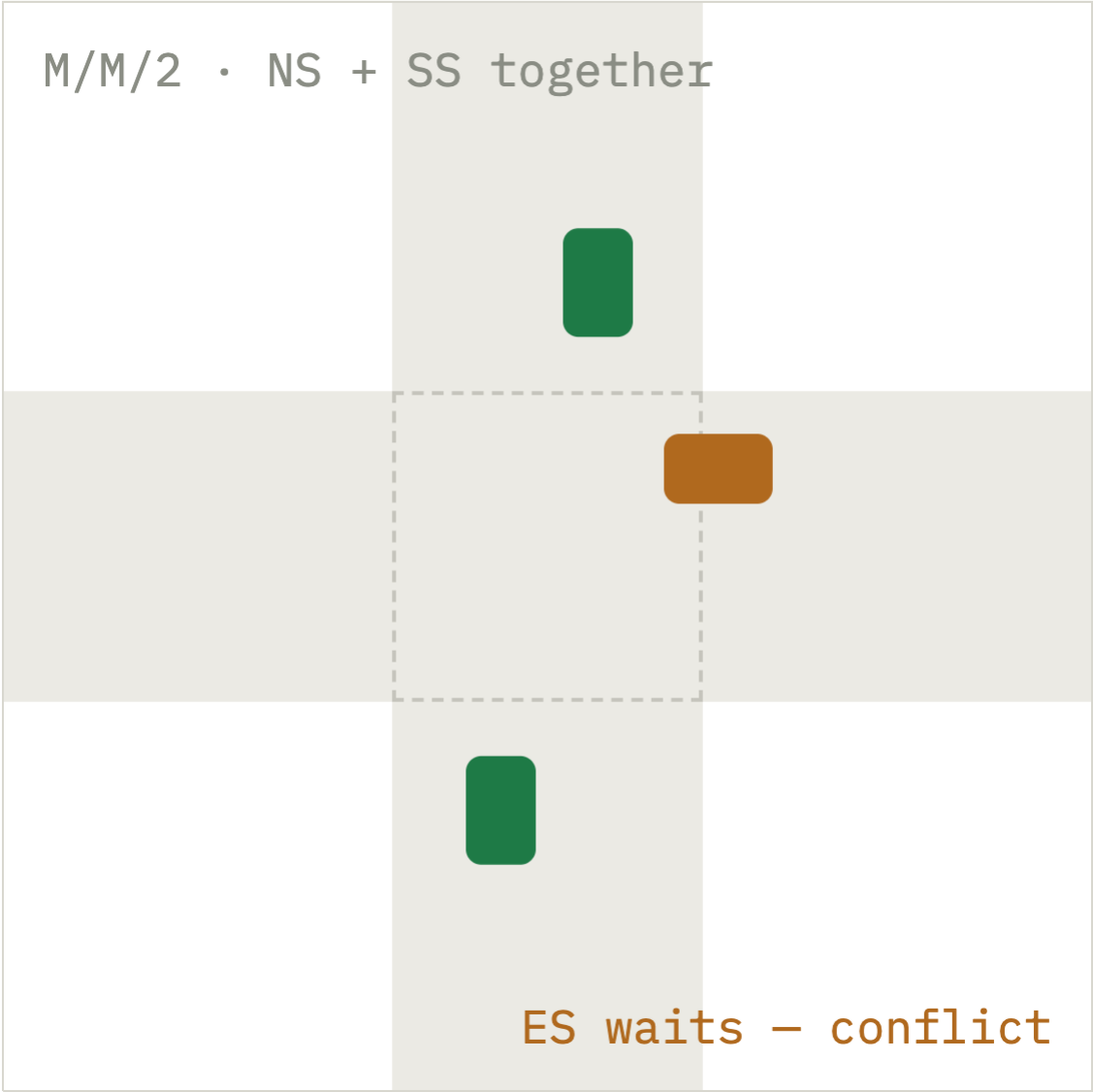
CommandTask	<code>prio 2</code>	non-blocking stdin reader — turns text commands into into queue messages
--------------------	---------------------	--

CrossWorker × 4	<code>prio 1</code>	fixed pool — times each crossing, allow enter / exit, frees the channel
------------------------	---------------------	---

Live Demo



- Every car on screen exists because a FreeRTOS task said so — request, grant, enter, exit.
- Controls send one-line commands: ADD, CH 1|2, CROSS, PAUSE, RESET.
- The raw engine stream is visible live in the event-log panel.



HW/SW Codesign

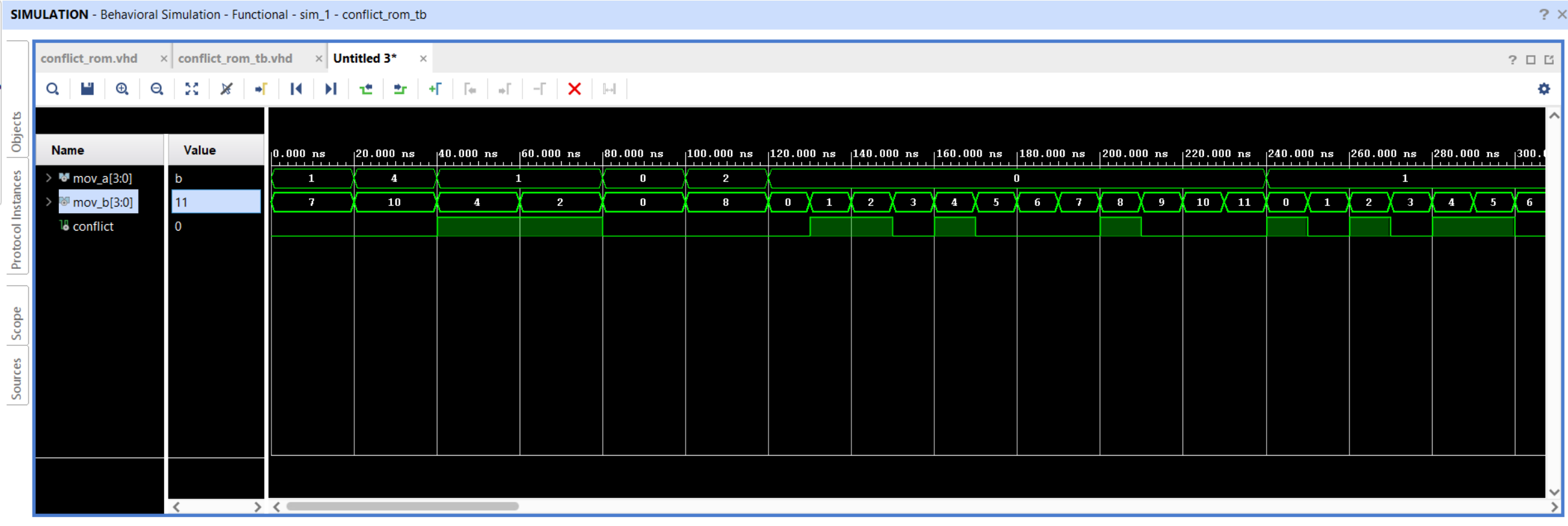
The critical operation — “do A and B conflict?” — moved into hardware. Conflicts shows a rise.

7/7 SATISFIED

M/M/1 & M/M/2 RUN

ALL 144 Lookups Correct

C == UPPAAL == VHDL



Both are generated from the same source, so software and hardware give identical answers by construction — Tested.

HW/SW Codesign

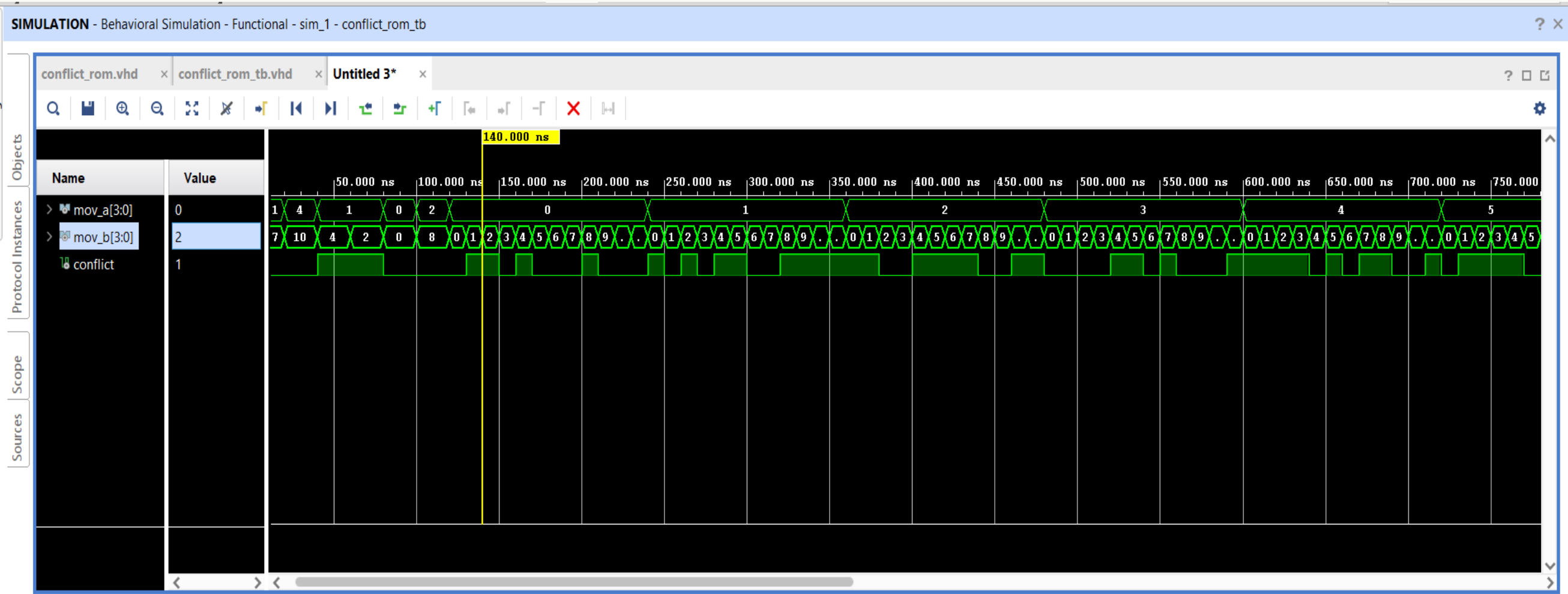
Zoomed out view.

THE HARDWARE MODULE

conflict_checker

```
in  mov_a      : 4-bit code
in  mov_b      : 4-bit code
out conflict : std_logic
```

- Combinational ROM — answers in one cycle, same latency for every pair.
- One conflict matrix generates both the C header and the VHDL ROM.
- $12 \times 12 = 144$ pairs simulated; hardware matches software exactly.



Simulation

The intersection (drawn from live FreeRTOS events)

waiting

crossing

highlighted pair

SAFE ✓ (no conflicting cars crossing, occupancy ≤ channels)

Controls → sent to FreeRTOS

Channels (queuing model) — sends CH to the engine

M/M/1 (one at a time)

M/M/2 (two if compatible)

Add a car — sends ADD <movement>

From South

turn Right

+ Add car

+ Random

☒ Auto-spawn (browser sends ADDs)

Arrival rate

Crossing time — sends CROSS

0.5 s

Pause engine

Reset

FIFS QUEUE (FROM ENGINE)

NOW CROSSING (CHANNELS)

SL

car 312

66%

STATS (AUTHORITATIVE, FROM FREERTOS)

Queue length

0

Occupancy (crossing)

1 / 1

Total served

313

Thank You.
